Assignment-3

ELL-786 MULTIMEDIA SYSTEMS

A report presented for the Assignment-3 on

Implementation of DPCM to compress Audio Signals

Submitted By:

Sudhanshu Chaudhary (2019JTM2207)
Santosh Kumar (2019JTM2088)

Bharti School Of

Telecommunication Technology and Management

IIT, Delhi

April 18, 2021

Contents

1 Problem Statement **3**

2 Theory **4**
2.1 Audio Digitization 4
2.2 DPCM 4
2.3 Prediction in DPCM 5
2.4 Golomb coding 6
2.4.1 Golomb Encoder Algorithm 6
2.4.2 Golomb Decoder Algorithm 6

3 Implementation **7**
3.1 Algorithm 7
3.2 Python Function Created: 8
3.3 Screenshots 9
3.3.1 Audio 1: Music 1 9
3.3.2 Audio 2: Dialogue 11
3.3.3 Audio 3: Speech 11
3.3.4 Audio 4: Music 2 14

4 Conclusion **15**

5 Appendix **16**
5.1 Appendix-A : Code 16

List of Figures

1 Audio Digitization 4
2 DPCM encoder and decoder 5
3 Predictor Coefficients 5
4 Golomb Encoder 6
5 Golomb Decoder 6
6 DPCM and Golomb compression 7
7 Choose input audio file 9
8 Music audio 9
9 Bit-rate, SNR, Prediction Error and SPER 10
10 SNR vs No. of Bits per sample 10
11 Voice audio 11
12 Bit-rate, SNR, Prediction Error and SPER 11
13 SNR vs No. of Bits per sample 12
14 Speech audio 12
15 Prediction coefficients 13
16 Bit-rate, SNR, Prediction Error and SPER 13
17 SNR vs No. of Bits per sample 13
18 Music audio 14
19 Bit-rate, SNR, Prediction Error and SPER 14
20 SNR vs No. of Bits per sample 15

1 Problem Statement

This is a programming assignment that requires you to implement DPCM which is followed by coding the prediction error.

Input: A .wav file for signalling input.

Coding Environment: Matlab/Python/C/C++ without using built-in libraries.

Experiment:

- Develop an algorithm using the DPCM technique for encoding and decoding audio signal (16 bit/Sample)
- Use a Linear Predictor of order N=1;2;4
- Use any coding technique of your choice to predict Prediction error.
- Compute the Rate and SNR for a different combination of predictors and number of quantization levels using different kinds of audio signal(voice, music of different genre).

2 Theory

2.1 Audio Digitization

- To get audio or video into a computer, we must digitize it.
- Audio signals are continuous in time and amplitude.
- Audio signal must be digitized in both time and amplitude to be represented in binary form.

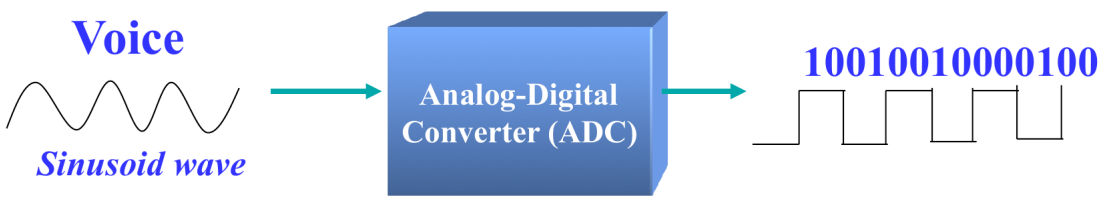

Figure 1: Audio Digitization

- For Analog-to-Digital conversion (ADC) to result in a faithful reproduction of the signal, slices, called samples, of the analog waveform must be taken frequently.
- The number of samples per second is called the sampling rate or sampling frequency.
- A Signal to Noise Ratio (SNR) is the ratio of the relative sizes of the signal values and the errors.
- The higher the SNR, the smaller the average error is with respect to the signal value, and the better the fidelity.

2.2 DPCM

Samples that are highly correlated, when encoded by PCM technique, leave redundant information behind. To process this redundant information and to have a better output, it is a wise decision to take a predicted sampled value, assumed from its previous output and summarize them with the quantized values. Such a process is called as Differential PCM (DPCM) technique. The DPCM Transmitter consists of Quantizer and Predictor with two summer circuits.

The predictor produces the assumed samples from the previous outputs of the transmitter circuit. The input to this predictor is the quantized versions of the input signal.
The notation of the signals is the same as the previous ones. In the absence of noise, the encoded receiver input will be the same as the encoded transmitter output.

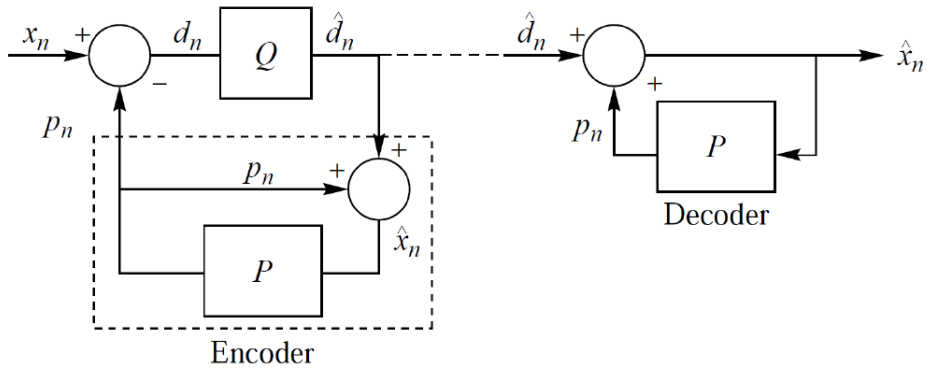

Figure 2: DPCM encoder and decoder

As mentioned before, the predictor assumes a value, based on the previous outputs. The input given to the decoder is processed and that output is summed up with the output of the predictor, to obtain a better output.

2.3 Prediction in DPCM

Differential encoding systems like DPCM gain their advantage by the reduction in the variance and dynamic range of the difference sequence. How much the variance is reduced depends on how well the predictor can predict the next symbol based on the past reconstructed symbols. Predictor Coefficients are calculated using auto-correlation function and Wiener-Hopf equations.

$$\mathbf{RA} = \mathbf{P}$$

$$\mathbf{R} = \begin{bmatrix} R_{xx}(0) & R_{xx}(1) & R_{xx}(2) & \cdots & R_{xx}(N-1) \\ R_{xx}(1) & R_{xx}(0) & R_{xx}(1) & \cdots & R_{xx}(N-2) \\ R_{xx}(2) & R_{xx}(1) & R_{xx}(0) & \cdots & R_{xx}(N-3) \\ \vdots & \vdots & & & \vdots \\ R_{xx}(N-1) & R_{xx}(N-2) & R_{xx}(N-3) & \cdots & R_{xx}(0) \end{bmatrix}$$

$$\mathbf{A} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_N \end{bmatrix} \qquad \mathbf{P} = \begin{bmatrix} R_{xx}(1) \\ R_{xx}(2) \\ R_{xx}(3) \\ \vdots \\ R_{xx}(N) \end{bmatrix}$$

$$\mathbf{A} = \mathbf{R}^{-1}\mathbf{P}.$$

Figure 3: Predictor Coefficients

2.4 Golomb coding

Golomb coding is a lossless data compression method, generally used for encoding natural numbers. Golomb coding uses a tunable parameter m to divide an input value n into two parts: q, the result of a division by m, and r, the remainder. The quotient is sent in unary coding, followed by the remainder in truncated binary encoding.

2.4.1 Golomb Encoder Algorithm

1. $k \leftarrow \lceil \log_2 (m) \rceil$.
2. $r \leftarrow s \mod m$.
3. $t \leftarrow 2^k - m$.
4. Output $(s \text{ div } m)$ using an unary code.
5. If $r < t$:
   a. Output the integer encoded in the $k-1$ least significant bits of r using a binary code.
6. Else:
   a. $r \leftarrow r + t$.
   b. Output the integer encoded in the k least significant bits of r using a binary code.

Figure 4: Golomb Encoder

2.4.2 Golomb Decoder Algorithm

1. $k \leftarrow \lceil \log_2 (m) \rceil$.
2. $t \leftarrow 2^k - m$.
3. Let $s \leftarrow$ the number of consecutive ones in the input (we stop when we read a 0).
4. Let $x \leftarrow$ the next $k-1$ bits in the input.
5. If $x < t$:
   a. $s \leftarrow s \times m + x$.
6. Else:
   a. $x \leftarrow x \times 2 + $ next input bit.
   b. $s \leftarrow s \times m + x - t$.

Figure 5: Golomb Decoder

3 Implementation

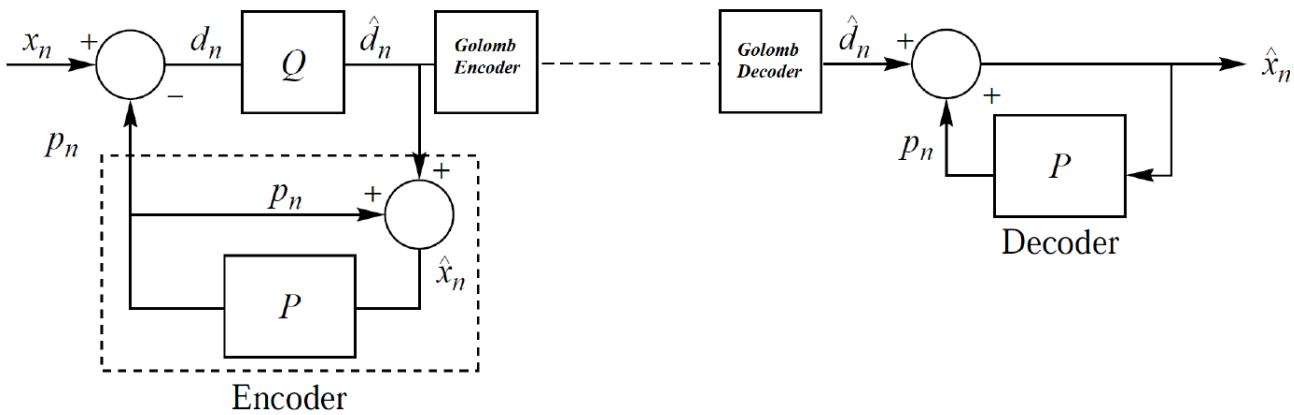

Figure 6: DPCM and Golomb compression

3.1 Algorithm

- Firstly, import the library needed.
- Read the file audio file and convert the values in INT16.
- Now We have a data X(n) so applied the Linear prediction algorithm.
- From LPA we calculated the prediction coefficient for different value of N=1,2,4.
- Now We have prediction coefficient(a1 for N=1, a1 a2 for N=2, a1 a2 a3 a4 for N=4) and data X(n).
- In iterative way we have calculated the predicted value for different N.
- Now Encoding starts here where we do D(n) = X(n)-P(n) and Quantized the value D(n) and save that value.
- In the same step we add D_Quantized+P(n) and give it to Prediction Function for next sequence prediction.
- The above two steps iterates throughout data form n=0 to n=endOfData.
- Now we have a Qunatized Difference of Data D_Quantized.
- The Difference value is now distributed as a two side geometric distribution.
- Now D_Quantized value is given to the **GOLOMB Encoder**.

- Encoded Value is Transmitted.
- At the Receiver end the received data is passed through **GOLOMB Decoder**. .
- The Decoded Output is now change to its normal form from Geometric Distribution to uniform.
- Now we have actual difference value so at DPCM Decoder the P(n) predicted value added to the difference.
- The Resultant value is Quantized value of actual data.
- From this data calculated the SNR(signal to Noise ratio) and SNPER (signal-to-prediction-error ratio).
- Also the Bit-rate calculated for different level of quantizer and Prediction Error Calculated.
- All Calculation is Done for three type of audio file(Speech,Music and Voice).
- The Different Level of Quantizer is used to see the change in parameter.

3.2 Python Function Created:

The main file for this assignment is A3.py in this all functions are created and used.

- uniformQuantizer(): Qunatizer function.
- predictorCoeff() Predictor Coefficient Calculation.
- predictorEncoder(): Input side Prediction encoder .
- predictorDecoder(): Output side Prediction decoder.
- encoderDPCM(): Normal DPCM Encoder without Prediction.
- decoderDPCM(): Normal DPCM Decoder without Prediction.
- write(): Output file writing function.
- SNR_PE(): SNR and Prediction Error Calculation function
- golombEncoder(): Golomb Encoder.
- golombDecoder(): Golomb Decoder.
- sign_of_data(): Sign of data Calculation function.
- outPredict(): Output Data prediction Quantized Function.
- Gmap(): Geometric distribution of Quantized Data.
- GmapInverse(): Geometric distribution Inverse Calculation of Golomb decoder Data.
- GolamParameterM(): Golomb Parameter M Calculation.

3.3 Screenshots

```
------------Multimedia Assignment 3---------

Choose the file:
1: Music file
2: Dialogue file
3: Speech file
4: Music file
Enter input:
1
Channels: 1
Framerate: 16000
Time: 3.0
```

Figure 7: Choose input audio file

3.3.1 Audio 1: Music 1

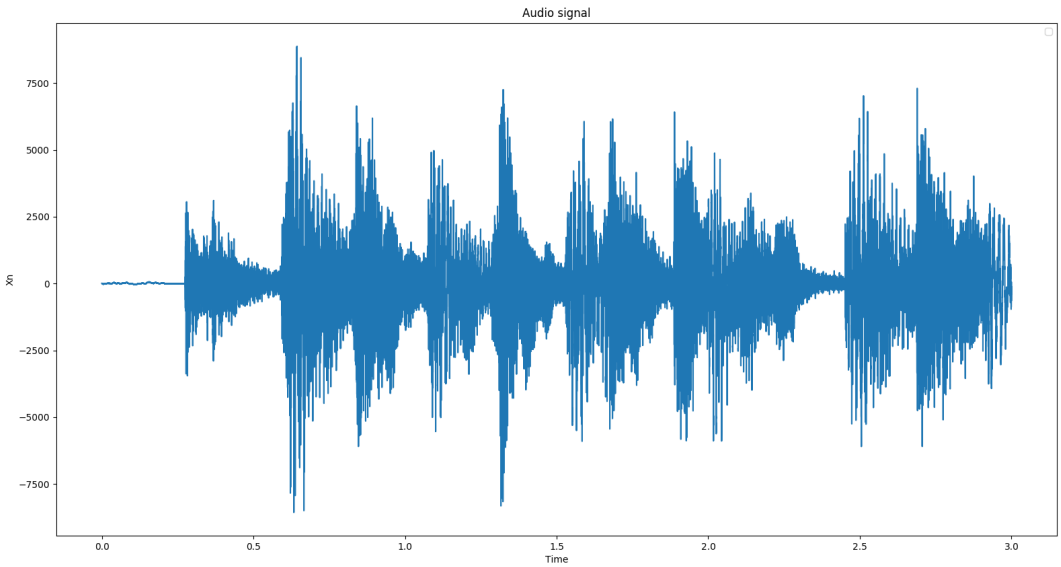

Figure 8: Music audio

```
################ Prediction Coefficient  ################

Predictor coefficients for N=1: [0.2396]
Predictor coefficients for N=2: [0.2196, 0.0834]
Predictor coefficients for N=4: [0.2133, 0.0681, 0.0629, 0.0132]

No_Bit_per_samples: 4
Bit Rate(kbps): 64

################ The Prediction Error ################
Prediction Error for N=1: 90030109124.93114
Prediction Error for N=2: 90025517869.61575
Prediction Error for N=4: 90023484777.13185

################ SNR(Signal to Noise Ratio) ################
SNR for N=1: 5.7529307195123565
SNR for N=2: 6.170682451710739
SNR for N=4: 6.356246029510809

####### SPER( Signal-to-prediction-error ratio)###########
SPER for N=1: 1.000289569304339
SPER for N=2: 1.0003405835601928
SPER for N=4: 1.0003631752753084
```

Figure 9: Bit-rate, SNR, Prediction Error and SPER

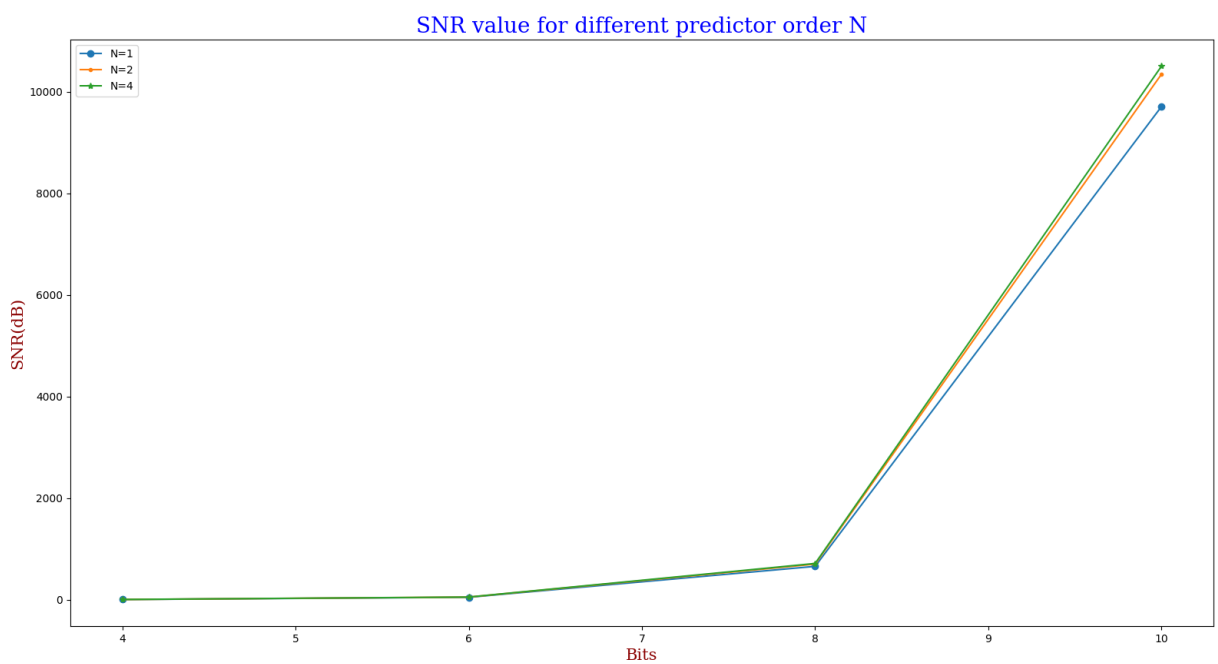

Figure 10: SNR vs No. of Bits per sample

3.3.2 Audio 2: Dialogue

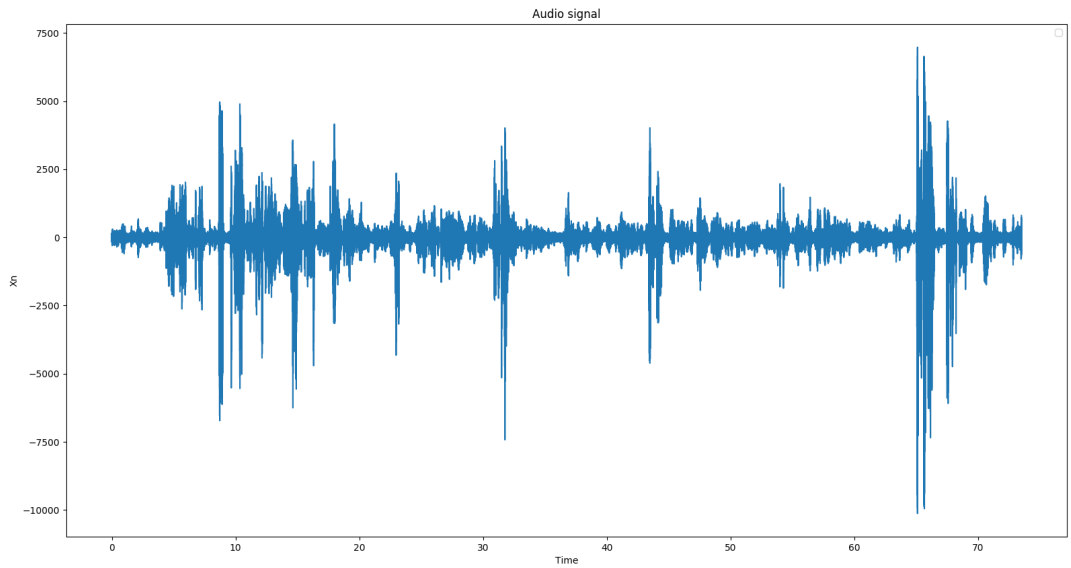

Figure 11: Voice audio

```
################ Prediction Coefficient  ################

Predictor coefficients for N=1: [0.9434]
Predictor coefficients for N=2: [1.1232, -0.1906]
Predictor coefficients for N=4: [1.1289, -0.2317, 0.1137, -0.0823]

No_Bit_per_samples: 4
Bit Rate(kbps): 64

################ The Prediction Error ################
Prediction Error for N=1: 225501737598.253
Prediction Error for N=2: 225496298147.69037
Prediction Error for N=4: 225504282439.5836

################ SNR(Signal to Noise Ratio) ################
SNR for N=1: 45.8110504530125
SNR for N=2: 60.306736396334294
SNR for N=4: 60.432442855943776

####### SPER( Signal-to-prediction-error ratio)###########
SPER for N=1: 1.0002691630201765
SPER for N=2: 1.00029329164094
SPER for N=4: 1.000257874869547
```

Figure 12: Bit-rate, SNR, Prediction Error and SPER

3.3.3 Audio 3: Speech

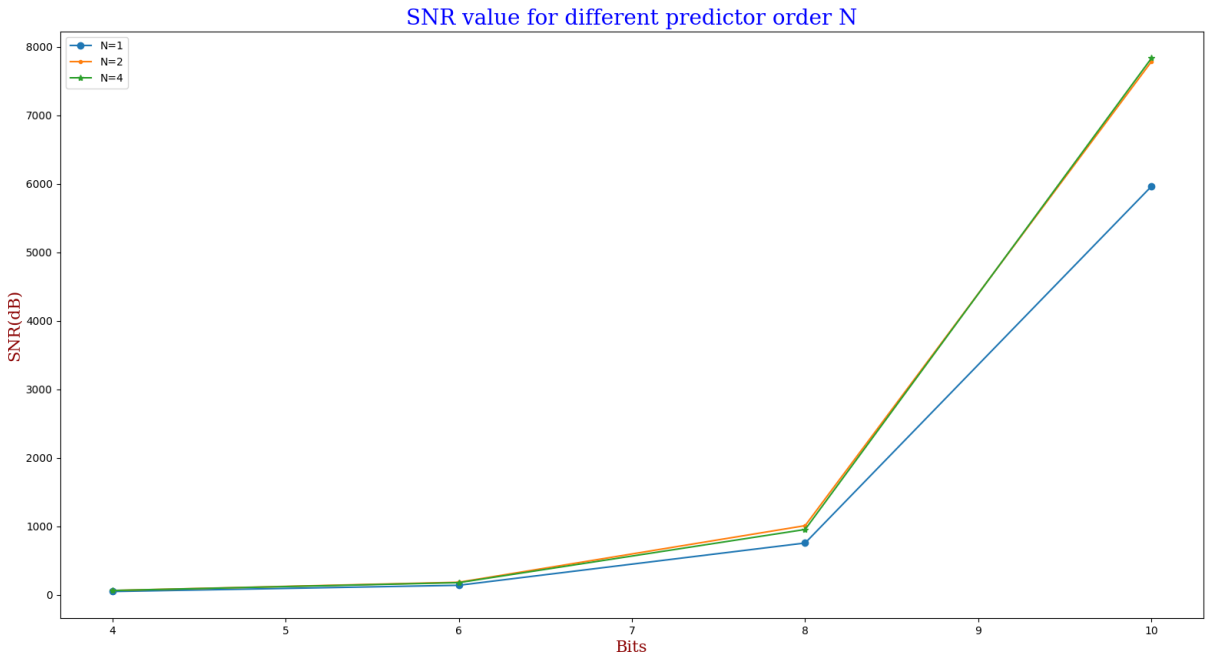

Figure 13: SNR vs No. of Bits per sample

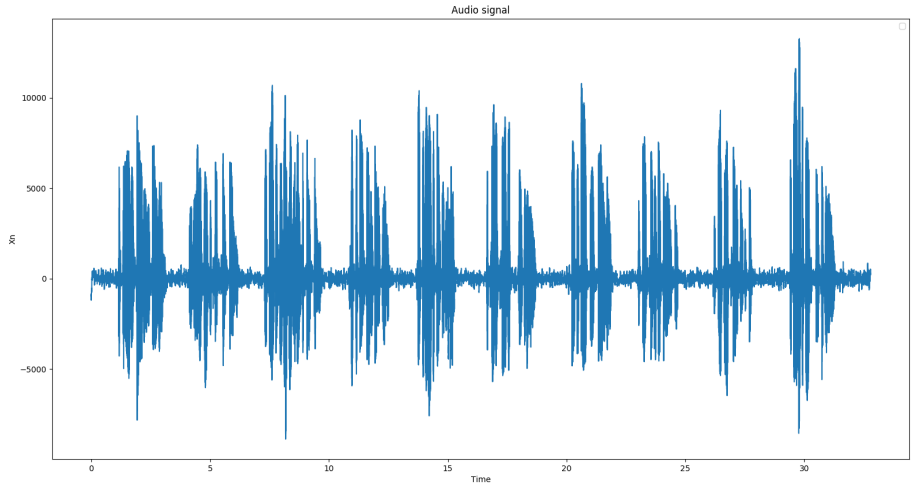

Figure 14: Speech audio

```
################ Prediction Coefficient  ################

Predictor coefficients for N=1: [0.9081]
Predictor coefficients for N=2: [0.6649, 0.2678]
Predictor coefficients for N=4: [0.6108, 0.1534, -0.0, 0.1876]
```

Figure 15: Prediction coefficients

```
No_Bit_per_samples: 8
Bit Rate(kbps): 128

################ The Prediction Error ################
Prediction Error for N=1: 1061509575036.6783
Prediction Error for N=2: 1059907217118.6349
Prediction Error for N=4: 1059907217118.6349

################ SNR(Signal to Noise Ratio) ################
SNR for N=1: 1598.3471551612392
SNR for N=2: 1954.0392309234992
SNR for N=4: 2838.8299484828003

####### SPER( Signal-to-prediction-error ratio)###########
SPER for N=1: 1.0049263093007343
SPER for N=2: 1.006445547591361
SPER for N=4: 1.006445547591361
```

Figure 16: Bit-rate, SNR, Prediction Error and SPER

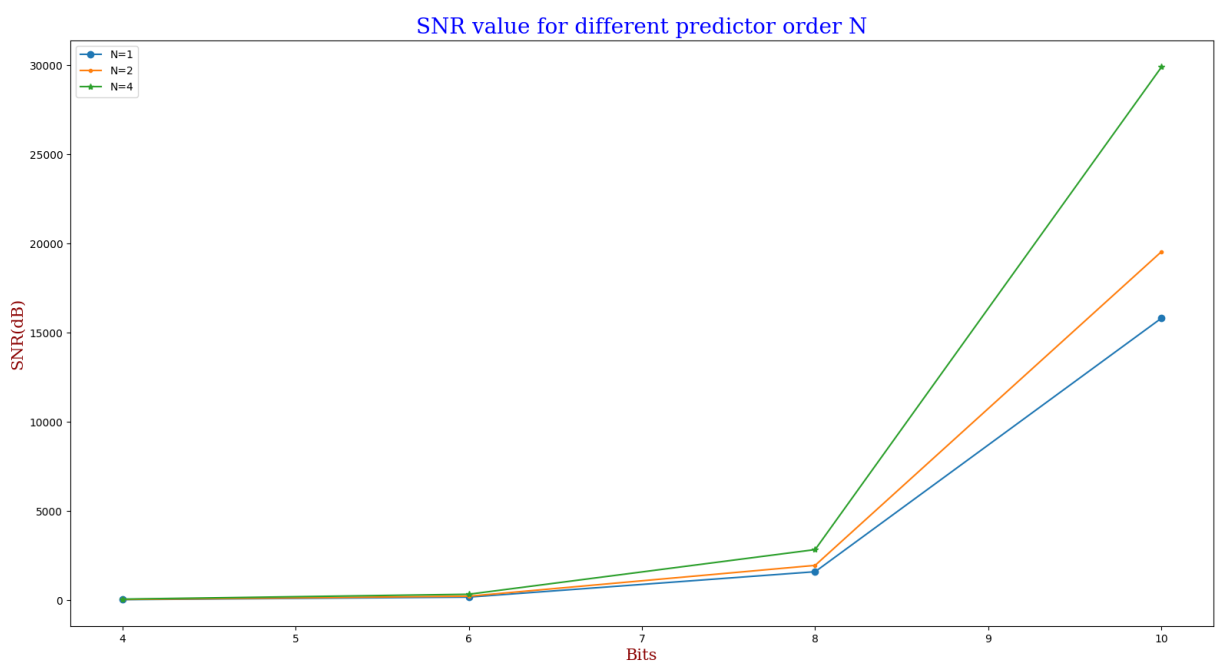

Figure 17: SNR vs No. of Bits per sample

3.3.4 Audio 4: Music 2

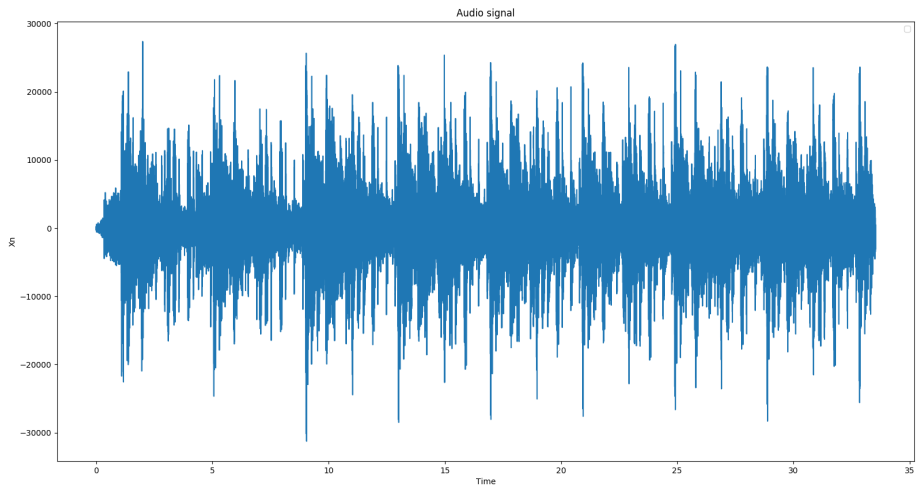

Figure 18: Music audio

```
################ Prediction Coefficient  ################

Predictor coefficients for N=1: [0.4141]
Predictor coefficients for N=2: [0.4784, -0.1551]
Predictor coefficients for N=4: [0.4879, -0.2082, 0.1089, 0.0942]

No_Bit_per_samples: 4
Bit Rate(kbps): 64

################ The Prediction Error ################
Prediction Error for N=1: 12700075944367.506
Prediction Error for N=2: 12700561914923.08
Prediction Error for N=4: 12699689798134.596

################ SNR(Signal to Noise Ratio) ################
SNR for N=1: 11.393620447189527
SNR for N=2: 8.749363079559103
SNR for N=4: 14.331824537335697

####### SPER( Signal-to-prediction-error ratio)###########
SPER for N=1: 1.0001800962180474
SPER for N=2: 1.00014182562181
SPER for N=4: 1.000210507651911
```

Figure 19: Bit-rate, SNR, Prediction Error and SPER

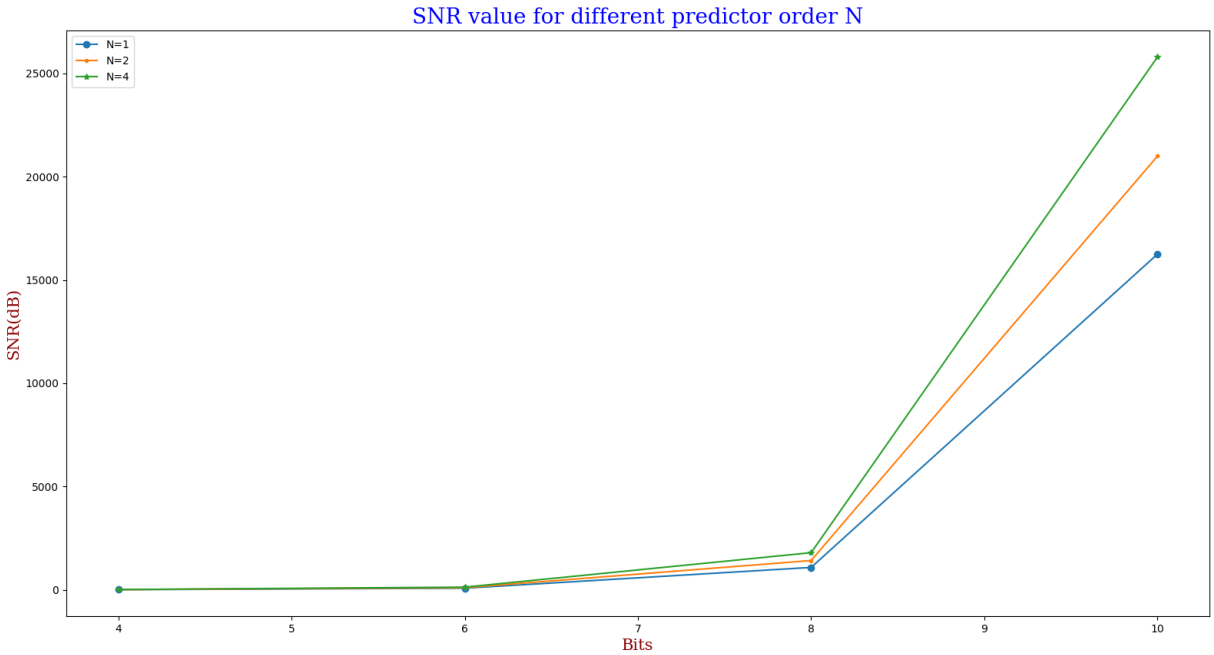

Figure 20: SNR vs No. of Bits per sample

4 Conclusion

We have presented DPCM scheme for different audio input like music, voice and speech and analyzed the performance indicators (SNR, SPER, Bit-rate, and prediction error). In almost all the cases the DPCM with N = 4 show better performance in terms of SNR,SPER and prediction error as compare to N=1 and N=2. In some case difference is more as compare to other cases.

5 Appendix

5.1 Appendix-A : Code

```
print(" ————————Multimedia Assignment 3————————")
print()
# Library imported
import time
import math
import wave
import struct
import numpy as np
from datetime import timedelta
import matplotlib.pyplot as plt

# start timer to check time
start_time = time.monotonic()

##########################################################

# New Quantizer Design for Defference value before Golam Encoder
def uniformQuantizer(Xdata, Bit, Delta):
    Nsteps = 2 ** (Bit - 1) - 1
    Q_Data = np.multiply(
        np.sign(Xdata).astype("int16"),
        np.minimum(np.floor(np.abs(Xdata) / Delta), Nsteps).astype("int16"),
    )
    return Q_Data


# Functon for predicting coefficients
def predictorCoeff(x, N):
    m = len(x)
    R = np.zeros((N, N))
    temp = []
    for k in range(N + 1):
        total = 0
        for i in range(1, m - k):
            total += x[i] * x[i + k]
        total = total / (m - k)
        temp.append(total)
    # print(temp, "temp")
    P = np.array(temp[1:])
    P.shape = N, 1
    # print(P)
    temp = temp[0:N]
    l = 0
    for i in range(len(R)):
        for j in range(len(R[0]) - l):
            R[i][j + l] = temp[j]
        l += 1
    # print(R)
    l = 1
```

```
    for i in range(1, len(R)):
        for j in range(l):
            R[i][j] = temp[l - j]
        l += 1
    # print(R)
    R_inv = np.linalg.inv(R)
    A = np.matmul(R_inv, P)
    # print(A)
    A = A.T[0]
    A = [round(i, 4) for i in A]
    return A


# Function for differnce using predictor
def predictorEncoder(A, x):
    N = len(A)
    pn_list = [0]
    diff_list = [x[0]]
    x_Q = [(x[0])]
    for n in range(1, len(x)):
        pn = 0
        for i in range(1, N + 1):
            if (n - i) < 0:
                xQ = 0
            else:
                xQ = x[n - i]
            pn += A[i - 1] * xQ
        pn_list.append(pn)
        diffQ = x[n] - pn
        diff_list.append(diffQ)
        xq = diffQ + pn
        x_Q.append(xq)
    return diff_list, pn_list


# Function for output using quantized differnece as input
def predictorDecoder(A, d_q):
    N = len(A)
    pn_list = [0]
    x_Q = [(d_q[0])]
    for n in range(1, len(d_q)):
        pn = 0
        for i in range(1, N + 1):
            if (n - i) < 0:
                xQ = 0
            else:
                xQ = d_q[n - i]
            pn += A[i - 1] * xQ
        pn_list.append(pn)
        x_Q.append(d_q[n] + pn)
    # print(x_Q)
    return x_Q, pn_list


```

```python
# Function for encoder
def encoderDPCM(data):
    res = [data[0]]
    for i in range(1, len(data)):
        d_q = data[i] - data[i - 1]
        res.append(d_q)
    return res


# Function for decoder
def decoderDPCM(data):
    res = [(data[0])]
    for i in range(1, len(data)):
        temp = data[i] + res[i - 1]
        res.append(temp)
    return res


# Function for array to audio file
def write(data, path, s):
    sampleRate = s
    obj = wave.open(path, "w")
    obj.setnchannels(1)  # mono
    obj.setsampwidth(2)
    obj.setframerate(sampleRate)
    for i in data:
        d = struct.pack("<h", int(i * pow(2, 15)))
        obj.writeframesraw(d)
    obj.close()
    return


def SNR_PE(x, xi, p):
    m = len(x)
    num = 0
    deno = 0
    pe = 0
    for i in range(1, m):
        deno += pow((x[i] - xi[i]), 2)
        num += pow((x[i]), 2)
        pe += pow((x[i] - p[i]), 2)
    SNR = num / deno
    SPER = num / pe
    # print("SNR:", SNR, SPER, pe)
    return SNR, pe, SPER


def golombEncoder(n_list, m):
    golomb_code_list = []
    for n in n_list:
        n = int(n)
        q = n // m
        r = n % m
        quo = "1" * q + "0"
```

```
        b = math.floor(math.log2(m))
        k = 2 ** (b + 1) - m
        if r < k:
            rem = bin(r)[2:]
            l = len(rem)
            if l < b:
                rem = "0" * (b - l) + rem
        else:
            rem = bin(r + k)[2:]
            l = len(rem)
            if l < b + 1:
                rem = "0" * (b + 1 - l) + rem
        golomb_code = quo + rem
        golomb_code_list.append(golomb_code)
    # print(golomb_code_list)
    return golomb_code_list


def golombDecoder(code_list, m):
    s_list = []
    for code in code_list:
        k = math.ceil(math.log2(m))
        t = (2 ** k) - m
        s = 0
        for i in range(len(code)):
            if code[i] == "1":
                s += 1
            else:
                break
        x = code[i + 1 : i + k]
        x = int(x, 2)
        if x < t:
            s = s * m + x
            s_list.append(s)
        else:
            x = x * 2 + int(code[i + k :], 2)
            s = s * m + x - t
            s_list.append(s)
    # print(s_list)
    return s_list


def sign_of_data(data):
    if data > 0:
        return -1
    elif data == 0:
        return 0
    else:
        return 1


def outPredict(Encoded_data_val, Delta_val, pn_Tx_list, lenth_0f_dat):
    Quantized_out = np.zeros(lenth_0f_dat)
    D_rX = np.zeros(lenth_0f_dat)
```

```
    # D_rX= np.sum(np.multiply(Delta,Encoded_data_val),np.multiply(np.sign(Encoded_data_val),Delta/2))
    for i in range(1, lenth_0f_dat):
        D_rX[i] = (
            Delta_val * Encoded_data_val[i]
            + sign_of_data(Encoded_data_val[i]) * Delta_val / 2
        )
        Quantized_out[i] = pn_Tx_list[i] + D_rX[i]
    return Quantized_out


########################## Golomb Distributed Data Function ######################
def Gmap(Q_dif_value, lenth_0f_data):
    pDnch = np.zeros(lenth_0f_data)
    for i in range(len(Q_dif_value)):
        if Q_dif_value[i] < 0:
            pDnch[i] = 2 * abs(Q_dif_value[i]) - 1
        else:
            pDnch[i] = 2 * abs(Q_dif_value[i])
    return pDnch


def GmapInverse(Deco_out):
    lenth_0f_data = len(Deco_out)
    pDnch = np.zeros(lenth_0f_data)
    for i in range(len(Deco_out)):
        n = Deco_out[i]
        if n % 2 == 1:
            pDnch[i] = -(n + 1) / 2
        else:
            pDnch[i] = n / 2
    return pDnch


######################### Golomb Parameter M from given Data ######################
def GolamParameterM(D_data, length_of_data):
    A_Q_dif_value = np.unique(D_data)
    probaPV, bina = np.histogram(D_data, bins=A_Q_dif_value)
    prob_of_occurance = (probaPV) / (length_of_data)
    max_value = np.max(prob_of_occurance)
    find_index = np.where(np.isclose(prob_of_occurance, max_value))
    M = np.floor(-1 / math.log2(1 - prob_of_occurance[find_index[0][0] + 1])) + 1
    # print(M)
    return M


############### Reading audio file ################
print("Choose the file:")
print("1: Music file")
print("2: Word file")
print("3: Speech file")
print("4: Music file")
print("Enter input:")
```

```
x = input()
if x == "1":
    file = "0Music.wav"
elif x == "2":
    file = "1Word.wav"
elif x == "3":
    file = "2Speech.wav"
elif x == "4":
    file = "3Music.wav"
else:
    print("Invalid option")

path = "/home/sudhanshu/Downloads/4thSem/Multimedia/As3/4Dialogue.wav"
pathout = "/home/sudhanshu/Downloads/4thSem/Multimedia/As3/"

file = wave.open(path)
print("Channels:", file.getnchannels())
f = file.getframerate()
print("Framerate:", f)
duration = 1 / f
nf = file.getnframes()
t = nf / f
print("Time:", t)
t_seq = np.arange(0, t, duration)
data = file.readframes(-1)
wav_data_int16 = np.frombuffer(data, "Int16")
# wav_data_float32 = wav_data_int16.astype(np.float32)
# max_int16 = 2 ** 15  # Normalise float32 array so that values are between
    -1.0 and +1.0
# wav_data_float32_normalised = wav_data_float32 / max_int16
inputdata = wav_data_int16
print()

x = encoderDPCM(inputdata)
y = decoderDPCM(x)
# Function to plot
plt.plot(t_seq, wav_data_int16)
plt.xlabel("Time")
plt.ylabel("Xn")
plt.title("Audio signal")
plt.legend()
plt.show()


############## DPCM predictor coefficients  ################

a1 = predictorCoeff(inputdata, 1)  # N=1
a2 = predictorCoeff(inputdata, 2)  # N=2
a4 = predictorCoeff(inputdata, 4)  # N=4

print("################ Prediction Coefficient  ################")
print("                                                        ")
print("Predictor coefficients for N=1:", a1)
print("Predictor coefficients for N=2:", a2)
```

```
print("Predictor coefficients for N=4:", a4)
print(" ")

###### Quantizing the Difference value Before Golam #####
SNR_1_list = []
SNR_2_list = []
SNR_4_list = []
Bits_list = []
for i in range(4, 12, 2):
    print("No_Bit_per_samples:", i)
    print("Bit Rate(kbps):", int((f * i) / 1000))
    print()
    No_Bit_per_samples = i
    Bits_list.append(No_Bit_per_samples)

    # N=1
    d1, pn1 = predictorEncoder(a1, inputdata)
    Delta1 = (2 * abs(max(d1))) / (2 ** No_Bit_per_samples)
    Q_dif_value1 = uniformQuantizer(d1, No_Bit_per_samples, Delta1)
    length_of_data1 = len(Q_dif_value1)
    D_data1 = Gmap(Q_dif_value1, length_of_data1)
    Parameter_M1 = GolamParameterM(D_data1, length_of_data1)
    # print("The Golomb Paraameter M for this Data N=1:", Parameter_M1)
    # Golomb Encoding
    g_D_data1 = golombEncoder(D_data1, int(Parameter_M1))
    # Golomb Decoding
    g_decoded1 = golombDecoder(g_D_data1, int(Parameter_M1))
    g_rescaled1 = GmapInverse(g_decoded1)
    op1, pn_op1 = predictorDecoder(a1, g_rescaled1)
    Quan_op1 = outPredict(Q_dif_value1, Delta1, pn1, length_of_data1)
    SNR_1, pe_1, SPER_1 = SNR_PE(inputdata, Quan_op1, pn_op1)
    SNR_1_list.append(SNR_1)

    # N=2
    d2, pn2 = predictorEncoder(a2, inputdata)
    Delta2 = (2 * abs(max(d2))) / (2 ** No_Bit_per_samples)
    Q_dif_value2 = uniformQuantizer(d2, No_Bit_per_samples, Delta2)
    length_of_data2 = len(Q_dif_value2)
    D_data2 = Gmap(Q_dif_value2, length_of_data2)
    Parameter_M2 = GolamParameterM(D_data2, length_of_data2)
    # print("The Golomb Paraameter M for this Data N=2:", Parameter_M2)
    # Golomb Encoding
    g_D_data2 = golombEncoder(D_data2, int(Parameter_M2))
    # Golomb Decoding
    g_decoded2 = golombDecoder(g_D_data2, int(Parameter_M2))
    g_rescaled2 = GmapInverse(g_decoded2)
    op2, pn_op2 = predictorDecoder(a2, g_rescaled2)
    Quan_op2 = outPredict(Q_dif_value2, Delta2, pn2, length_of_data2)
    SNR_2, pe_2, SPER_2 = SNR_PE(inputdata, Quan_op2, pn_op2)
    SNR_2_list.append(SNR_2)

    # N=4
    d4, pn4 = predictorEncoder(a4, inputdata)
    Delta4 = (2 * abs(max(d4))) / (2 ** No_Bit_per_samples)
```

```
    Q_dif_value4 = uniformQuantizer(d4, No_Bit_per_samples, Delta4)
    length_of_data4 = len(Q_dif_value4)
    D_data4 = Gmap(Q_dif_value4, length_of_data4)
    Parameter_M4 = GolamParameterM(D_data4, length_of_data4)
    # print("The Golomb Paraameter M for this Data N=4:", Parameter_M4)
    # Golomb Encoding
    g_D_data4 = golombEncoder(D_data4, int(Parameter_M4))
    # Golomb Decoding
    g_decoded4 = golombDecoder(g_D_data4, int(Parameter_M4))
    g_rescaled4 = GmapInverse(g_decoded4)
    op4, pn_op4 = predictorDecoder(a4, g_rescaled4)
    Quan_op4 = outPredict(Q_dif_value4, Delta4, pn4, length_of_data4)
    SNR_4, pe_4, SPER_4 = SNR_PE(inputdata, Quan_op4, pn_op4)
    SNR_4_list.append(SNR_4)

    print("################ The Prediction Error ################")
    print("Prediction Error for N=1:", pe_1)
    print("Prediction Error for N=2:", pe_2)
    print("Prediction Error for N=4:", pe_4)
    print()
    print("################ SNR(Signal to Noise Ratio) ################")
    print("SNR for N=1:", SNR_1)
    print("SNR for N=2:", SNR_2)
    print("SNR for N=4:", SNR_4)
    print()
    print("####### SPER( Signal-to-prediction-error ratio)###########")
    print("SPER for N=1:", SPER_1)
    print("SPER for N=2:", SPER_2)
    print("SPER for N=4:", SPER_4)
    print()

# Function to plot
font1 = {"family": "serif", "color": "blue", "size": 20}
font2 = {"family": "serif", "color": "darkred", "size": 15}
plt.plot(Bits_list, SNR_1_list, marker="o", label="N=1")
plt.plot(Bits_list, SNR_2_list, marker=".", label="N=2")
plt.plot(Bits_list, SNR_4_list, marker="*", label="N=4")
plt.xlabel("Bits", fontdict=font2)
plt.ylabel("SNR(dB)", fontdict=font2)
plt.title("SNR value for different predictor order N", fontdict=font1)
plt.legend()
plt.show()

# write(x, pathout + "out.wav", f)
#########################################################
end_time = time.monotonic()
print()
print("Execution time: ", timedelta(seconds=end_time - start_time))
print()
#########################################################
```

References

[1] *Differential PCM.* https://www.tutorialspoint.com/digital_communication/digital_communication_differential_pcm.htm.

[2] *Differential pulse-code modulation.* https://en.wikipedia.org/wiki/Differential_pulse-code_modulation.

[3] *Differential (Predictive) Coding.* https://vicente-gonzalez-ruiz.github.io/differential_coding/.

[4] *Explain DPCM and ADPCM techniques used in audio compression.* https://www.ques10.com/p/7267/explain-dpcm-and-adpcm-techniques-used-in-audio--1/?

[5] *Golomb coding.* https://w3.ual.es/~vruiz/Docencia/Apuntes/Coding/Text/03-symbol_encoding/09-Golomb_coding/index.html.